Agentic MMIX Pipeline — Low-Level Design & System Documentation

Part 4: Agentic Framework & Agent Configuration

---

1. Framework & Tools

1.1 Technology Stack

Component	Technology	Version	Purpose
LLM	Azure OpenAI GPT-4.1	API 2024-12-01-preview	Quality evaluation (structured JSON), narrative generation (5 sections)
State Management	Custom `AgentState` class	N/A	Tracks pipeline state, decisions, trace, spec strategy
Orchestration	Custom state machine	N/A	Pipeline phase management, iterative loop-back, strategy escalation
Quality Evaluation	Three-layer hybrid (rules + LLM + logic)	N/A	Determines PROCEED vs RETRY with HARD/SOFT criteria
Modeling	statsmodels, scikit-learn	0.14+, 1.3+	6 linear regression algorithm types
Data Processing	pandas, numpy	2.0+, 1.24+	All data manipulation
Visualization	matplotlib, seaborn	3.7+, 0.12+	EDA, diagnostic, and scenario plots

1.2 Design Decision: Custom vs LangChain/CrewAI

The orchestrator uses a custom state machine rather than LangChain, CrewAI, or AutoGen because:

1. **Well-defined linear flow:** The pipeline has a clear sequence (load → EDA → clean → aggregate → features → model → evaluate → retry?) with one decision point (post-modeling evaluation). LangChain adds abstraction overhead for what is essentially a while loop with a conditional.
2. **Full trace control:** The reasoning trace format is custom — every decision includes timestamp, phase, iteration, reasoning text, decision string, and a details dict. Framework-imposed trace formats would require adaptation.
3. **No external dependencies:** Beyond the OpenAI SDK, the orchestrator has zero framework dependencies. This simplifies deployment and debugging.

4. **LangGraph compatibility:** The architecture maps directly to LangGraph if migration is desired:
   - Each `tool_*` function → LangGraph node
   - `evaluate_and_decide()` → conditional edge
   - `AgentState` → LangGraph state schema
   - `spec_strategy` → state-dependent routing
5. **Single-agent design:** This pipeline uses one agent (the orchestrator) with one LLM (GPT-4.1 for evaluation). Multi-agent frameworks like CrewAI add overhead for a single-agent use case.

1.3 LLM Usage

The LLM (GPT-4.1) is called in two distinct contexts:

Context	Function	Temperature	Max Tokens	Calls per Run
Quality evaluation	`QualityEvaluator.llm_evaluate()`	0.1 (focused)	800	1-3 (one per iteration)
Narrative generation	`NarrativeGenerator.narrate_*()`	0.3 (creative)	1500	5 (one per section)

Total LLM calls per pipeline run: **6-8** (evaluation is cheap; narratives are the bulk).

---

2. Agent Architecture

2.1 Components

Component	Class/Function	Purpose
AgentState	`AgentState` class	Tracks ALL pipeline state: data, results, decisions, trace, spec_strategy, adjustments
QualityEvaluator	`QualityEvaluator` class	Three-layer evaluation: rule-based HARD + LLM with EDA context + decision logic
Pipeline Tools	8 `tool_*` functions	Callable functions for each pipeline phase (load, eda, outliers, aggregate, features, model, scenarios, narratives)
Decision Engine	`evaluate_and_decide()`	Orchestrates the PROCEED/RETRY decision with full reasoning capture
Improvement Suggester	`suggest_improvements()`	Maps quality issues to concrete strategy changes on state

2.2 AgentState Class

```python
class AgentState:
    # Pipeline outputs — populated by tool functions
    data: Dict[str, DataFrame]        # Phase 1: 7 raw DataFrames
    clean_data: Dict[str, DataFrame]   # Phase 3: after outlier cleaning
    outlier_log: List[Dict]           # Phase 3: cleaning action log
    assumptions: Dict                 # Phase 3: A1-A9 assumptions dict
    corr_matrix: pd.DataFrame         # Phase 2: correlation matrix (feeds LLM context)
    aggregated_data: Dict             # Phase 4: weekly or monthly modeling data
    fe_result: Dict                   # Phase 5: feature matrix + specs + VIF
    model_result: Dict                # Phase 6: ranked models, convergence, best
    narrator: NarrativeGenerator      # Phase 9: narrative generator instance

    # Agent tracking — recorded every decision
    current_phase: str                # e.g. "modeling", "quality_evaluation"
    iteration: int                    # 0 = pre-loop, 1+ = iteration number
    max_iterations: int               # Default 3
    reasoning_trace: List[Dict]       # Full timestamped decision log
    decisions: List[str]              # Flat list of decision strings
    quality_scores: Dict[int, Dict]   # Per-iteration: {score, issues, suggestions, reasoning}

    # Agentic strategy — controls pipeline behavior
    spec_strategy: str                # "base", "expanded", "groups", "monthly"
    adjustments: Dict                 # Current iteration's adjustment details
    last_llm_verdict: Dict            # Most recent LLM evaluation response
```

Key design: `spec_strategy` is the primary lever the agent uses to change behavior between iterations. It's set by `suggest_improvements()` based on quality issues and/or LLM feedback.

2.3 Pipeline Tools

Tool	Phase	Critical?	Failure Behavior	Special Behavior
`tool_load_data(state)`	1	YES	Pipeline stops	Stores 7 DataFrames on state.data
`tool_run_eda(state)`	2	NO	Continue without EDA	Uses `data=state.data` (no double load)
`tool_run_outliers(state)`	3	YES	Pipeline stops	Stores clean_data, log, assumptions

Tool	Phase	Critical?	Failure Behavior	Special Behavior
`tool_run_aggregation(state)`	4	YES	Pipeline stops	Can be re-called for monthly fallback
`tool_run_features(state)`	5	YES	Pipeline stops	**Filters specs by state.spec_strategy**
`tool_run_modeling(state, skip_scenarios=True)`	6	YES	Pipeline stops	Scenarios skipped during loop
`tool_run_scenarios(state)`	8	NO	Continue without	**Post-loop only** — on approved model
`tool_run_narratives(state)`	9	NO	Continue without	**Post-loop only** — passes agent_context

`tool_run_features` spec filtering logic:

```python
BASE_SPECS = {A, B, C, D, E, F}      # 6 base specs (3 features each)
WEEKLY_SPECS = {G, H, I, J}          # 4 weekly specs (6-8 features each)
GROUP_SPECS = {A, J}                 # Group-only specs (for VIF issues)

if strategy == "base":
    filtered = specs matching BASE_SPECS        # 6 specs
elif strategy == "expanded":
    filtered = specs matching WEEKLY_SPECS      # 4 specs (delta only)
elif strategy == "groups":
    filtered = specs matching GROUP_SPECS       # 2 specs
elif strategy == "monthly":
    filtered = specs matching BASE_SPECS        # 6 specs (on monthly data)
```

---

3. Iteration Strategy

3.1 Progressive Escalation

The agent uses a **progressive escalation strategy** — it starts conservative and adds complexity only when the simpler approach fails. This is more efficient than running all specs on every iteration and mirrors how a human analyst would work.

Iteration	Strategy	Specs	Models	Granularity	R^2 Threshold	Rationale
1	**base**	A-F (6)	72	weekly (47 pts)	≥ 0.50	Start simple: 3-feature specs. If the data has strong signals, simple models will find them.
2	**expanded** (delta)	G-J only (4)	48	weekly (47 pts)	≥ 0.50	Add group-based 6-8 feature specs. NOT re-running A-F (already proven in iter 1). Compare best vs iter 1 best.
3	**monthly** (fallback)	A-F (6)	72	monthly (10 pts)	≥ 0.70	Different granularity as last resort. Monthly aggregates away weekly noise. Higher R^2 threshold because fewer points = easier to overfit.

3.2 Delta Approach (Iteration 2)

On iteration 2, the agent runs ONLY the new specs (G-J) that weren't tested in iteration 1. This avoids re-running the 72 models from iteration 1.

After modeling, the agent compares:

- Best model from iteration 2 (from specs G-J)
- Best model from iteration 1 (from specs A-F, stored as `prev_best`)

The winner (higher R^2) is kept as the final best model. This means iteration 2 can only improve or maintain — it can never make things worse.

```python
if current_r2 > prev_r2:
    state.add_reasoning("model_comparison",
        f"Iter 2 R²={current_r2:.3f} > iter 1 R²={prev_r2:.3f}. Keeping new model.",
        "IMPROVED")
else:
    state.model_result["best_model"] = prev_best  # Restore previous
    state.add_reasoning("model_comparison",
        f"Iter 2 R²={current_r2:.3f} <= iter 1 R²={prev_r2:.3f}. Keeping previous.",
        "NO_IMPROVEMENT")
```

3.3 Monthly Fallback (Iteration 3)

If iteration 2 (expanded weekly specs) still doesn't satisfy quality criteria:

1. Agent sets `state.spec_strategy = "monthly"` and `state.granularity = "monthly"`
2. `tool_run_aggregation()` is re-called, rebuilding the data at monthly granularity (10 data points)
3. `tool_run_features()` builds the feature matrix from monthly data — only base specs (A-F) are valid (weekly specs need n≥30)
4. `tool_run_modeling()` builds 72 candidates with the monthly threshold ($R^2 \geq 0.70$)

Monthly models often achieve higher R^2 (fewer data points, cleaner signal) but at the cost of model complexity (max 3 predictors) and statistical power.

3.4 Improvement Mapping

The `suggest_improvements()` function maps detected issues to concrete strategy changes:

Issue Detected	Current Strategy	New Strategy	Reasoning
Low R^2 (rule-based)	base	**expanded**	Base specs (3 features) explain too little variance. Adding group-based specs (6-8 features) for Ridge/Lasso.
Low R^2 (rule-based)	expanded	**monthly**	Expanded weekly still insufficient. Try different granularity.
High VIF	any	**groups**	Multicollinearity detected. Restrict to group-only specs (A, J).
Ordinality failure	any	strict_ordinality filter	Wrong coefficient signs. Filter models post-hoc.
Unstable (CV gap)	expanded	**base**	High-feature specs causing overfitting. Revert to simpler.
LLM RETRY, no rule issues	base	**expanded**	LLM identifies model as insufficient despite passing rules. Default escalation.
LLM RETRY, no rule issues	expanded	(no change)	Already at max weekly complexity. Iteration 3 will try monthly.

4. Quality Evaluation — Three-Layer System

4.1 Layer 1: Rule-Based HARD Criteria

Deterministic checks that cannot be overridden. If any HARD criterion fails, the model is rejected regardless of LLM opinion.

Check	Threshold	Issue Text	Suggestion
R^2 (weekly)	≥ 0.50	"Low R2 (X.XXX < 0.5)"	"Try different feature combinations or add more features"
R^2 (monthly)	≥ 0.70	"Low R2 (X.XXX < 0.7)"	"Try different feature combinations"
Adjusted R^2	≥ 0.45	"Low Adj R2 (X.XXX < 0.45)"	—
Ordinality	Convergence-based PASS	"Best model fails ordinality"	"Remove or combine collinear features"
VIF	< 50	"High VIF (XX > 50)"	"Use grouped channels instead of individual"
Models passed	≥ 5	"Only N models succeeded"	"Check feature availability"
CV stability	CV R^2/Train R^2 ≥ 0.5	"Unstable model (ratio = X.XX)"	"Reduce complexity or add regularization"

Score calculation:

```python
score = 1.0 - (n_issues × 0.15)
score = max(0, min(1, score))
acceptable = (no issues) OR (score >= 0.5 AND no "Low R2" issue)
```

4.2 Layer 2: LLM Evaluation (GPT-4.1)

The LLM receives a structured prompt with three types of context:

Static context (data constraints):

```
- This is a POC with only 12 months of data (Jul 2015 - Jun 2016)
- Weekly: 47 data points. Monthly: 10-11 data points.
- R-squared of 0.50-0.60 is realistic for weekly.
- For monthly, R² >= 0.70 is acceptable.
```

Dynamic context (from EDA — changes with data):

```
EDA FINDINGS:
- NPS has -0.96 correlation with GMV (strong negative). EDA determined
  this is a seasonality artifact. A negative NPS coefficient is EXPECTED.
- EDA found channels with NEGATIVE GMV correlation: Radio(-0.95), Other(-0.96).
- Convergence CONFIRMS negative impact for: log_spend_digital_brand,
  spend_digital_brand, nps_standardized, total_gmv_lag1.
```

The dynamic context is built by extracting from `state.corr_matrix`:

- NPS-GMV correlation (flags if < -0.5)
- Channels with GMV correlation < -0.3
- Convergence-confirmed negative features

This means if the pipeline runs on a different dataset where NPS is positive, the prompt automatically adjusts — no hardcoded assumptions.

HARD/SOFT criteria:

```
==========
HARD CRITERIA (non-negotiable — if ANY fails, verdict MUST be RETRY):
==========
1. Weekly R² >= 0.50 OR Monthly R² >= 0.70
2. CV MAPE < 5%
3. Ordinality PASS (convergence-based)
4. Sale features positive

Check these FIRST. If any fails, verdict = "RETRY" immediately.
Do not rationalize or override.

==========
SOFT CRITERIA (use judgment):
==========
- CV R² close to Train R² (stability)
- VIF < 10 preferred
- Coefficient magnitudes reasonable (0.01-0.5)
- If all HARD pass and soft mostly met → ACCEPT
```

Model data in prompt:

- Best model: spec, type, transform, R^2, Adj R^2, CV R^2, MAPE, ordinality, VIF
- Coefficients with values
- Convergence summary (top 6 features with direction and model count)
- Rule-based issues detected
- Iteration number, strategy, specs evaluated

Response format:

```json
{
  "verdict": "ACCEPT" or "RETRY",
  "reasoning": "3-5 sentences. Realistic about data constraints.",
  "coefficient_assessment": "Do elasticities align with convergence and EDA?",
  "suggestions": ["specific actionable change if RETRY"]
}
```

4.3 Layer 3: Decision Logic

```python
```

```
# Priority order matters:
if max_iterations_reached:
    decision = "PROCEED"        # Accept best available — can't iterate further

elif llm_verdict == "RETRY":
    decision = "RETRY"          # LLM respected even if rules pass
                                # (rules may have low thresholds but LLM
                                #  applies domain judgment)

elif not rule_based_acceptable:
    decision = "RETRY"          # Rules always enforced (HARD criteria)

else:
    decision = "PROCEED"        # Both layers approve
```

Key design decision: LLM RETRY is respected even when rule-based passes. This is because:

1. Rule-based thresholds are necessarily coarse (single R^2 cutoff)
2. The LLM considers coefficient reasonableness, convergence quality, stability gap, and domain knowledge
3. If the LLM sees a problem the rules miss (e.g., suspicious coefficient patterns), it should be able to trigger a retry

However, the max_iterations guard ensures the LLM cannot cause infinite retries. After 3 iterations, the pipeline proceeds with the best available model.

---

5. Narrative Prompt Templates

5.1 System Prompt (All Narratives)

```
You are a senior marketing analytics consultant specializing in Marketing Mix
Modeling (MMIX) for e-commerce. You explain data findings in clear, actionable
business language for marketing directors and C-suite executives.

Rules:
- Be concise but insightful (3-5 sentences per section unless more detail needed)
- Structure as: What happened → Why it matters → What to do about it
- Use business language, not statistical jargon
- When mentioning numbers, round to 1 decimal and use Cr (Crores) for currency
- Highlight actionable recommendations
```

- Flag risks and limitations honestly
- Do not make up numbers — only use what is provided

5.2 EDA Narrative

Analyze these EDA findings from an e-commerce Marketing Mix dataset...
DATA OVERVIEW: {data_summary}
CHANNEL CORRELATIONS: {corr_summary}
SALE EVENT IMPACT: {sale_lift}%
Write a 2-3 paragraph executive summary covering:
1. Revenue drivers
2. Channel effectiveness
3. Risks

5.3 Outlier Narrative

Explain these data cleaning decisions for a marketing stakeholder...
CLEANING ACTIONS: {log_text}
KEY ASSUMPTIONS: {assumption_text}
Write covering:
1. What was cleaned and why
2. What was intentionally kept (and why)
3. Any risks from these decisions

5.4 Feature Narrative (with multicollinearity context)

Explain these feature engineering steps...
STEPS: {steps_text}
MULTICOLLINEARITY: {vif_summary} ← per-spec VIF max and high-count
CHANNEL RANKING: {channel_ranking} ← GMV correlation per channel
Note: Some channels are highly correlated (r=0.99)...
Write covering:
1. Features created and why each matters
2. Channel grouping logic
3. Why groups vs individual channels (multicollinearity)
4. Transformations and business interpretation
5. Limitations

5.5 Modeling Narrative (with agent context)

Interpret these results for a business audience...

```
BEST MODEL: {metrics}
COEFFICIENTS: {elasticities}
CONVERGENCE: {consensus directions}
VIF: {best model multicollinearity}
MODEL SELECTION NOTE: Only linear models used (no tree-based — no elasticities)
AGENT HISTORY: {iterations, quality progression, strategy changes}
Write covering:
1. Marketing effectiveness
2. Channel impact (interpret elasticities)
3. Confidence (R², CV, convergence)
4. Limitations and caveats
```

The agent context block is only included if multiple iterations occurred:

```
AGENT ITERATION HISTORY:
  The agentic pipeline ran {n} iterations before accepting this model.
  Final strategy: {strategy}
  Quality progression:
    Iteration 1: score=0.70, issues=[Low R2]
    Iteration 2: score=1.00, issues=[]
  Adjustments: expanded to group-based specs
```

5.6 Scenario Narrative

```
Translate these scenario simulations into actionable recommendations...
SCENARIOS: {results with GMV changes}
Write covering:
1. Best single channel increase
2. Whether budget reallocation is worth it
3. Sale events vs media spend — which lever is stronger
4. Recommended budget strategy
5. Risks of the recommended strategy
```

---

6. Decision Flow Diagrams

6.1 Main Pipeline Flow

```
START
 │
 ├── Load data (Phase 1) ─── fail? → STOP
```

```
├── EDA (Phase 2) ──────────── fail? → continue (non-critical)
├── Clean data (Phase 3) ── fail? → STOP
├── Aggregate (Phase 4) ─── fail? → STOP
│
├── ┌── ITERATION 1 (strategy=base) ──────────────────────┐
│   │ Features: filter to A-F (6 specs, 3 features)      │
│   │ Modeling: 72 candidates, skip scenarios            │
│   │ Two-pass rank: initial → convergence → re-rank     │
│   │ Evaluate: HARD rules + LLM + decision logic        │
│   │ R²=0.47 < 0.50 → RETRY                             │
│   └────────────────────────────────────────────────────┘
│     │
│     ├── suggest_improvements → spec_strategy = "expanded"
│     │
│   ┌── ITERATION 2 (strategy=expanded, delta) ──────────┐
│   │ Features: filter to G-J only (4 specs, 6-8 feats)  │
│   │ Modeling: 48 NEW candidates only                   │
│   │ Compare best vs iter 1 best → keep winner          │
│   │ Evaluate: HARD + LLM                               │
│   │ R²=0.58 ≥ 0.50 → ACCEPT (if LLM agrees)            │
│   └────────────────────────────────────────────────────┘
│     │                         │
│     │ (if ACCEPT)             │ (if RETRY)
│     │                         │
│     │                 ┌── ITERATION 3 (monthly fallback) ──┐
│     │                 │ Re-aggregate to monthly (10 pts)   │
│     │                 │ Features: A-F, threshold R²≥0.70   │
│     │                 │ 72 candidates                      │
│     │                 │ Max iterations → PROCEED anyway    │
│     │                 └────────────────────────────────────┘
│     │                         │
│     ▼                         ▼
├── Scenarios (Phase 8) ── on approved model only
├── Narratives (Phase 9) ── with agent iteration context
└── Save reasoning trace → DONE
```

6.2 Quality Evaluation Flow

```
evaluate_and_decide(state, evaluator):
  │
  ├── Rule-based evaluation
  │   ├── Check R² ≥ 0.50 (weekly) or ≥ 0.70 (monthly)
  │   ├── Check Adj R² ≥ 0.45
```

```
│   ├── Check ordinality (convergence-based)
│   ├── Check VIF < 50
│   ├── Check n_models ≥ 5
│   ├── Check CV stability ≥ 0.5
│   └── Score = 1.0 - (n_issues × 0.15)
│
├── LLM evaluation (GPT-4.1)
│   ├── Build dynamic EDA context (NPS, negative channels, convergence)
│   ├── Send HARD + SOFT criteria with model data
│   └── Parse JSON response → verdict, reasoning, suggestions
│
└── Decision logic:
    │
    ├── max_iterations reached?
    │   └── YES → PROCEED (accept best available)
    │
    ├── LLM says RETRY?
    │   └── YES → RETRY (respected even if rules pass)
    │
    ├── Rules say NOT acceptable?
    │   └── YES → RETRY
    │
    └── Both pass → PROCEED
```

---

7. Reasoning Trace Format

7.1 Trace Entry Structure

Each decision is recorded as a dict appended to `state.reasoning_trace`:

```python
```

```
{
  "phase": str,        # e.g. "modeling", "llm_evaluation", "final_decision"
  "iteration": int,    # 0 = pre-loop, 1+ = iteration number
  "reasoning": str,    # Human-readable explanation of the decision
  "decision": str,     # PROCEED, RETRY, STOP, EVALUATE, IMPROVED, etc.
  "details": Dict,     # Additional context — varies by phase:
                       #   modeling: {r2, n_models}
                       #   rule_based: {score, issues, suggestions}
                       #   llm_evaluation: {coefficient_assessment, suggestions}
                       #   retry: {adjustments, new_strategy, issues}
                       #   feature_engineering: {n_specs, strategy, specs_used}
                       #   model_comparison: {prev_r2, new_r2, improvement}
  "timestamp": str     # HH:MM:SS
}
```

7.2 Example Trace — Two-Iteration Run (Actual)

```
AGENT REASONING TRACE
============================================================
 [19:36:07] init (iter 0)
   Reasoning: Starting agentic pipeline: weekly, models=all.
              Iteration 1 will use base specs (3 features).
              Agent will expand to high-feature specs if R² is insufficient.
   Decision:  PROCEED

 [19:36:11] data_loading (iter 0)
   Reasoning: Loaded 7 datasets
   Decision:  PROCEED

 [19:36:16] eda (iter 0)
   Reasoning: EDA complete. 4 issues found.
   Decision:  PROCEED

 [19:36:17] outlier_detection (iter 0)
   Reasoning: Cleaning complete. 3 actions taken.
   Decision:  PROCEED

 [19:37:03] aggregation (iter 0)
   Reasoning: weekly aggregation: 47 periods.
   Decision:  PROCEED
   n_periods: 47
   granularity: weekly
```

[19:37:04] feature_engineering (iter 1)
Reasoning: Strategy: base. Base specs only (6 specs, max 3 features each).
Matrix: 47 rows × 46 columns.
Decision: PROCEED
n_specs: 6
strategy: base
specs_used: [A, B, C, D, E, F]

[19:38:18] modeling (iter 1)
Reasoning: Best model: spec_B_total_spend (Huber) R2=0.465
Decision: EVALUATE
r2: 0.465
n_models: 60

[19:38:18] rule_based_evaluation (iter 1)
Reasoning: Model quality NEEDS IMPROVEMENT. Score: 0.70.
R2=0.465, Adj R2=0.427. 2 issues found.
Decision: ISSUES_FOUND
score: 0.70
issues: [Low R2 (0.465 < 0.5), Low Adj R2 (0.427 < 0.45)]

[19:38:23] llm_evaluation (iter 1)
Reasoning: The model's R-squared of 0.47 falls below the 0.50 HARD
threshold. While the positive coefficients for total investment
and sale events align with convergence, the model needs more
features to capture sufficient variance.
Decision: RETRY
suggestions: [Expand to group-based specs with more features]

[19:38:23] final_decision (iter 1)
Reasoning: LLM recommends RETRY. Rule-based score=0.70 (issues found).
Both layers agree: model needs improvement.
Decision: RETRY

[19:38:23] retry (iter 1)
Reasoning: Agent adjusting strategy: expanded.
Reason: Base specs (3 features) explain too little variance.
Adding group-based specs (6-8 features) for Ridge/Lasso.
Decision: RETRY
new_strategy: expanded
issues: [Low R2, Low Adj R2]

[19:38:24] feature_engineering (iter 2)
Reasoning: Strategy: expanded. Delta: 4 new high-feature specs only

(6-8 features). Matrix: 47 rows × 46 columns.
Decision: PROCEED
n_specs: 4
strategy: expanded
specs_used: [G, H, I, J]

[19:39:10] modeling (iter 2)
Reasoning: Best model: spec_H_groups_with_momentum (Ridge) R2=0.582
Decision: EVALUATE
r2: 0.582
n_models: 40

[19:39:10] model_comparison (iter 2)
Reasoning: Iter 2 best R2=0.582 > prev best R2=0.465.
Keeping new model.
Decision: IMPROVED
prev_r2: 0.465
new_r2: 0.582

[19:39:10] rule_based_evaluation (iter 2)
Reasoning: Model quality ACCEPTABLE. Score: 1.00.
R2=0.582, Adj R2=0.520. 0 issues found.
Decision: ACCEPTABLE
score: 1.00
issues: []

[19:39:15] llm_evaluation (iter 2)
Reasoning: The model meets all HARD criteria: R^2=0.58 exceeds the 0.50
threshold, CV MAPE is low at 1.4%, and ordinality passes with
convergence-confirmed directions. Traditional and digital
performance spend show expected positive effects. The negative
digital brand coefficient aligns with convergence evidence.
Decision: ACCEPT

[19:39:15] final_decision (iter 2)
Reasoning: Quality acceptable. Score=1.00. R2=0.582. LLM also approved.
Decision: PROCEED

[19:39:30] scenarios (iter 2)
Reasoning: Ran 11 scenarios on approved model (spec_H).
Top impact: +47.2%.
Decision: PROCEED

[19:39:55] narratives (iter 2)

Reasoning: Narratives generated with agent context
(2 iterations, base→expanded strategy).
Decision: PROCEED

7.3 Trace Entries by Phase

Phase	When Recorded	Key Details
init	Pipeline start	Granularity, model filter, starting strategy
data_loading	Phase 1	Number of datasets loaded
eda	Phase 2	Number of issues found
outlier_detection	Phase 3	Number of cleaning actions
aggregation	Phase 4	Granularity, number of periods
feature_engineering	Phase 5 (each iter)	Strategy, n_specs, specs_used, matrix shape
modeling	Phase 6 (each iter)	Best spec, model type, R^2, n_models
model_comparison	After iter 2+	prev_r2, new_r2, improvement
rule_based_evaluation	Phase 7 (each iter)	Score, issues list, suggestions
llm_evaluation	Phase 7 (each iter)	Verdict, reasoning, coefficient_assessment, suggestions
final_decision	Phase 7 (each iter)	Combined decision with full reasoning
retry	If RETRY	New strategy, adjustments dict, issues that triggered
scenarios	Phase 8	Number of scenarios, top impact %
narratives	Phase 9	Confirmation with agent context mention

8. Output Artifacts

Artifact	Location	Format	Generated By	Phase
7 EDA plots	outputs/plots/*.png	PNG	eda_pipeline.py	2
Outlier summary	outputs/plots/outlier_summary.png	PNG	outlier_detection.py	3

Artifact	Location	Format	Generated By	Phase
Feature distributions	outputs/plots/feature_distributions.png	PNG	feature_engineering.py	5
Features vs target	outputs/plots/features_vs_target.png	PNG	feature_engineering.py	5
Model rankings	outputs/plots/model_rankings.png	PNG	modeling_engine.py	6
Best model diagnostics	outputs/plots/best_model_diagnostics.png	PNG	modeling_engine.py	6
Scenario results	outputs/plots/scenarios.png	PNG	modeling_engine.py	8 (post-loop)
Narrative report	outputs/reports/mmix_narrative_report.md	Markdown	narrative_generator.py	9
Reasoning trace	outputs/reports/agent_reasoning_trace.txt	Text	agent_orchestrator.py	Final

8.1 Narrative Report Sections

The `mmix_narrative_report.md` contains 5 sections, each generated by a separate LLM call:

Section	Content	LLM Context Includes
Exploratory Data Analysis	Revenue drivers, channel effectiveness, risks	Data summary, channel correlations, sale lift
Data Quality & Cleaning	What cleaned, what kept, risks	Cleaning log, assumptions A1-A9
Feature Engineering	Features, grouping, multicollinearity, limitations	VIF summary, channel ranking, grouping rationale
Model Results	Effectiveness, channel impact, confidence, caveats	Coefficients, convergence, VIF, agent iteration history
Scenario Recommendations	Best channel, reallocation, sale vs spend, strategy	All scenario results with GMV changes

8.2 Reasoning Trace File

The `agent_reasoning_trace.txt` is a human-readable dump of `state.get_trace_summary()`. It includes every entry from `state.reasoning_trace` formatted as:

```
[TIMESTAMP] PHASE (iter N)
  Reasoning: HUMAN_READABLE_TEXT
  Decision:  DECISION_STRING
  key1: value1
  key2: value2
```

This file is the primary evidence of agentic behavior for evaluation. It shows:

- The agent started with a conservative strategy
- Detected quality issues (with specific metrics)
- Reasoned about improvements (both rule-based and LLM)
- Applied concrete changes (strategy escalation)
- Verified improvement (model comparison)
- Only ran scenarios and narratives after approval